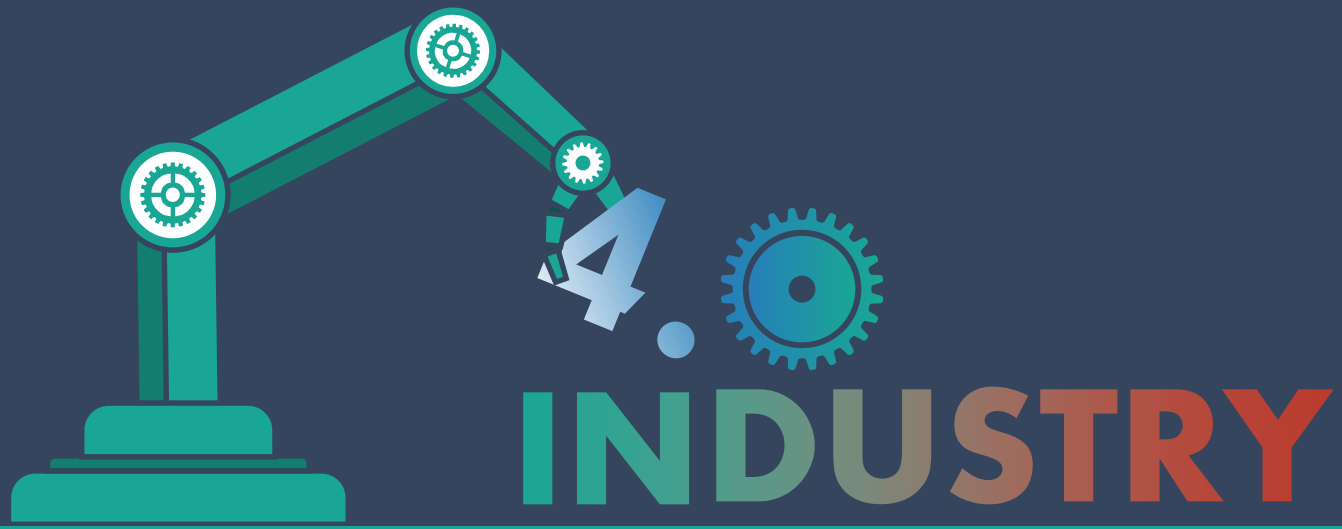
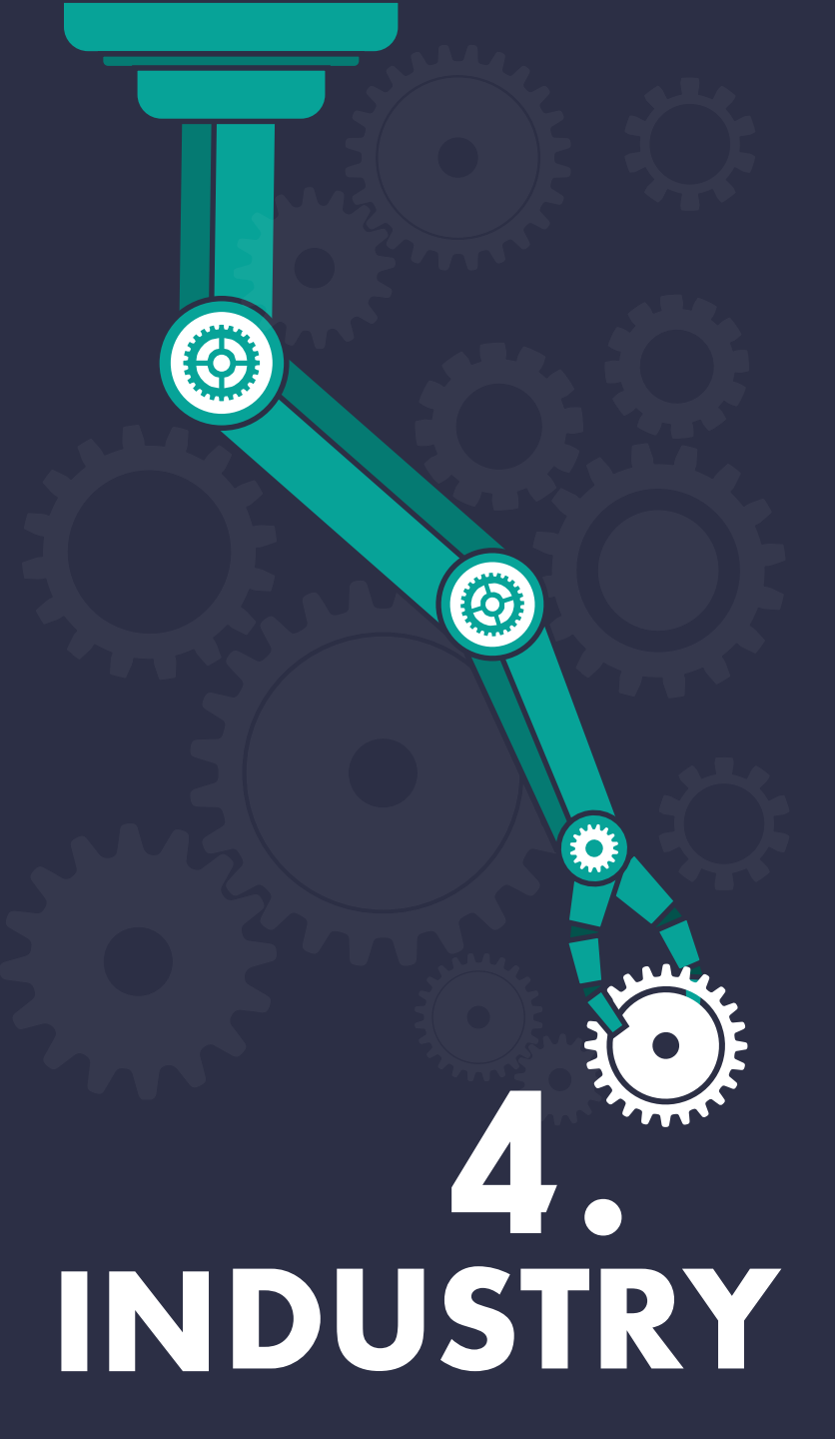


Node-RED

Modbus



M2M 機聯網



Agenda

01 **Node-Red**

物聯網可視化平台介紹與操作

02 **Modbus**

ASCII，RTU，ModbusTCP協定介紹

03 **Modbus 實習**

Write: Fun05, Fun15 , Fun06 , Fun16 **Read:** Fun01, Fun03

04 **機台監控**

機電丙生產資訊監控

What is Node-RED

Low-code programming for event-driven applications

Node-RED is a programming tool for wiring together hardware devices, APIs and online services in new and interesting ways.

Node-RED is built on Node.js, taking full advantage of its event-driven, non-blocking model. This makes it ideal to run at the edge of the network on low-cost hardware such as the Raspberry Pi as well as in the cloud.

Browser-based flow editing

Node-RED provides a browser-based flow editor that makes it easy to wire together flows using the wide range of nodes in the palette.
JavaScript functions can be created within the editor using a rich text editor.
A built-in library allows you to save useful functions, templates or flows for re-use.

Built on Node.js

The light-weight runtime is built on Node.js, taking full advantage of its event-driven, non-blocking model. This makes it ideal to run at the edge of the network on low-cost hardware such as the Raspberry Pi as well as in the cloud.

Social Development

The flows created in Node-RED are stored using JSON which can be easily imported and exported for sharing with others.

Install Node-RED

1. Install Node.js

Download the latest LTS version of Node.js from the official Node.js home page. It will offer you the best version for your system.

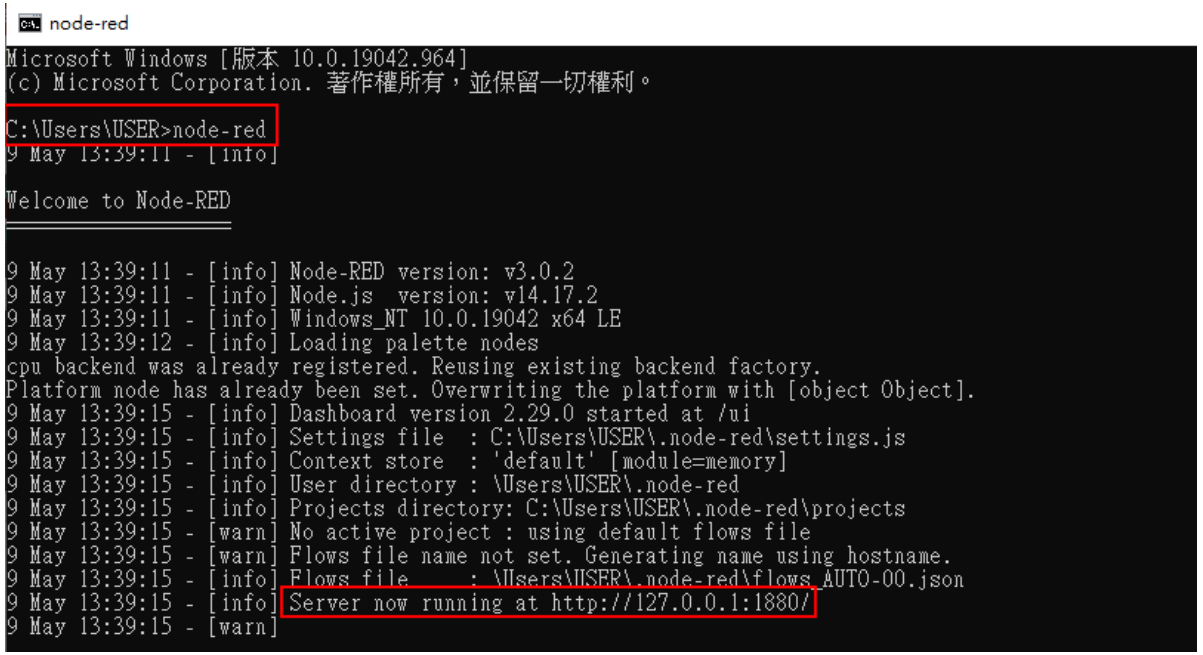
2. Install Node-RED

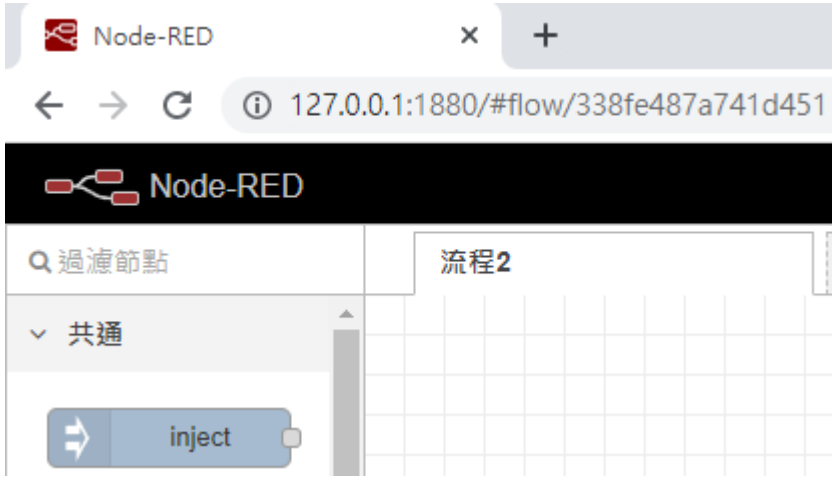
Installing Node-RED as a global module adds the command node-red to your system path. Execute the following at the command prompt:

```
npm install -g --unsafe-perm node-red
```

3.Run Node-RED

Once installed, you are ready to run Node-RED.





Getting Started

Node-RED Concepts https://nodered.org/docs/user-guide/concepts

The Core Nodes https://nodered.org/docs/user-guide/nodes

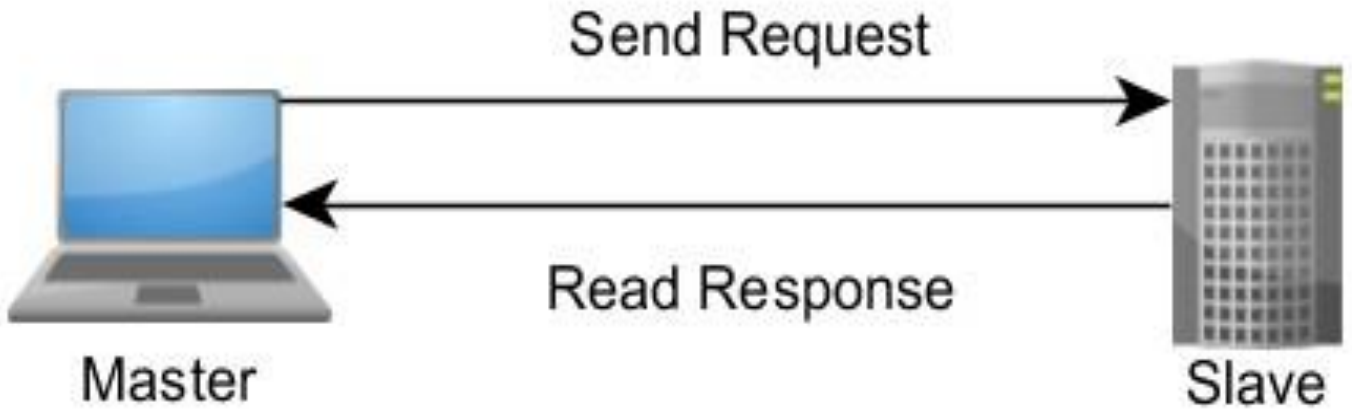
Adding nodes to the palette https://nodered.org/docs/user-guide/runtime/adding-nodes

Creating your first flow https://nodered.org/docs/tutorials/first-flow

Working with messages https://nodered.org/docs/user-guide/messages

Modbus通訊協定

Modbus 是一種需求-回應協定，採用主從架構(Master-Slave)。由Master主動提出需求，由Slave回應其需求。
Master-Slave 架構已經更名為 Client-Server架構。



PRESS RELEASE

Modbus Organization Replaces Master-Slave with Client-Server

July 9, 2020 — The Modbus Organization Board of Trustees announces it is expunging all occurrences of inappropriate language of the query and response paradigm of Modbus communications. With recent events, all companies and individuals with the ability to make positive changes should do so. As the holder of the standard for an industrial network with one of the largest installed bases in the world, it is important to complete the replacement of the current terminology with more thoughtful references. All instances of "master-slave" in the organization's literature and on its website will be removed. Current electronic content is being updated as of this announcement.

Modbus 常見問題解答：關於協議

什麼是 Modbus ® 協議？

Modbus協議是Modicon於1979年開發的一種消息結構，用於在智能設備之間建立客戶端-服務器通信。它是一個事實上的標準，真正開放，是工業製造環境中使用最廣泛的網絡協議。它已被數百家供應商在數千種不同設備上實施，以在控制設備之間傳輸離散/模擬 I/O 和寄存器數據。它是不同製造商之間的通用語言或公分母。一份報告稱其為“多供應商集成的實際標準”。行業分析師報告稱，僅在北美和歐洲就有超過 700 萬個 Modbus 節點。

Modbus ® 用在什麼地方？

Modbus 用於多個客戶端-服務器應用程序以監控和編程設備；實現智能設備與傳感器、儀器之間的通信；使用 PC 和 HMI 監控現場設備。
Modbus 還是需要無線通信的 RTU 應用的理想協議。因此，它被用於無數的天然氣和石油以及變電站應用中。但 Modbus 不僅僅是一種工業協議。建築、基礎設施、交通和能源應用也利用了它的優勢。

什麼是 Modbus TCP/IP 協議？

TCP/IP是互聯網通用的傳輸協議，實際上是一組分層協議，提供機器間可靠的數據傳輸機制。以太網已成為公司企業系統的事實標準，因此它也成為工廠網絡的事實標準也就不足為奇了。以太網不是一項新技術。它已經成熟到實施該網絡解決方案的成本已經下降到其成本與當今現場總線的成本相當的地步。
在工廠中使用以太網 TCP/IP 允許與支持工廠的企業內部網和 MES 系統真正集成。為了讓 Modbus 進入 21 世紀，1999 年開發了一個開放的 Modbus TCP/IP 規範。協議規範和實施指南可供下載（ www.modbus.org/specs ）。
Modbus 將多功能、可擴展且無處不在的物理網絡（以太網）與通用網絡標準 (TCP/IP) 和供應商中立的數據表示相結合，提供了一個真正開放、可訪問的過程數據交換網絡。對於任何支持 TCP/IP 套接字的設備來說，實現起來都很簡單。

https://modbus.org/faq.php

Modbus 常見問題解答：關於協議

為什麼要使用 Modbus TCP/IP？

在為您的設備選擇網絡時，Modbus TCP/IP 提供了幾個顯著的優勢：

- **簡單性：** Modbus TCP/IP 簡單地採用 Modbus 指令集並將 TCP/IP 包裹在它周圍。如果您已經擁有 Modbus 驅動程序並且了解以太網和 TCP/IP 套接字，則可以在幾個小時內啟動並運行驅動程序並與 PC 通信。開發成本非常低。需要最少的硬件，並且在任何操作系統下都可以輕鬆開發。

- **標準以太網：**不需要特殊的芯片組，您可以使用標準 PC 以太網卡與您新實施的設備通信。隨著以太網成本的下降，您將受益於硬件價格的下降並且隨著性能從 10 Mb 提高到 100 Mb 並很快提高到 1 Gb，您的技術也會隨之發展，從而保護您的投資。您不再依賴於一家供應商的支持，而是受益於成千上萬的開發人員，他們正在使以太網和互聯網成為未來的網絡工具。為 Modbus TCP/IP 協議分配眾所周知的以太網端口 502 恰如其分地補充了這項工作。

- **開放：** Modbus 協議於 2004 年 4 月從施耐德電氣轉移到 Modbus 組織，標誌著對開放的承諾。**該規範可免費下載**，使用 Modbus 或 Modbus TCP/IP 協議無需支付後續許可費用。Modbus TCP 工具包中提供了其他示例代碼、實施示例和診斷，這是 Modbus 組織成員的免費權益，非成員也可以購買
- **許多設備的可用性**：不同供應商設備之間的互操作性以及與大量安裝的 Modbus 兼容設備的兼容性使 Modbus 成為絕佳的選擇。

https://modbus.org/faq.php

Modbus 常見問題解答：關於協議

我可以在互聯網上使用 Modbus TCP/IP 嗎？

Modbus TCP/IP 是一種互聯網協議。TCP/IP 是 Internet 的傳輸協議這一事實自動意味著 Modbus TCP/IP 可以在 Internet 上使用。它旨在實現這一目標。實際上，這意味著安裝在歐洲的 Modbus TCP/IP 設備可以從世界任何地方通過 Internet 尋址。對設備供應商或最終用戶的影響是無窮無盡的。使用 PC 和瀏覽器對遠程設備執行維護和維修可降低支持成本並改善客戶服務。從家裡登錄到工廠的控制系統允許維護工程師最大限度地延長工廠的正常運行時間並減少現場時間。使用商業可用的 Internet/Intranet 技術，管理地理分佈式系統變得容易。

現有的 Modbus 設備能否通過 Modbus TCP/IP 進行通信？

由於 Modbus TCP/IP 只是帶有 TCP 包裝器的 Modbus 協議，因此現有的 Modbus 設備通過 Modbus TCP/IP 進行通信非常簡單。需要網關設備將當前的物理層（RS232、RS485 或其他）轉換為以太網，並將 Modbus 協議轉換為 Modbus TCP/IP。這種網關設備可以使用 PC 來實現。可以從幾個不同的製造商那裡獲得用於此目的的商業產品。Modbus 設備數據庫可以幫助您識別網關和其他 Modbus 設備。

我期望從 Modbus TCP/IP 系統獲得什麼樣的性能？

性能取決於網絡和硬件。如果您在 Internet 上運行 Modbus TCP/IP，您將不會比典型的 Internet 響應時間更好。但是，當出於調試和維護目的進行通信時，這可能就足夠了，您不必在星期天早上趕飛機或去現場！

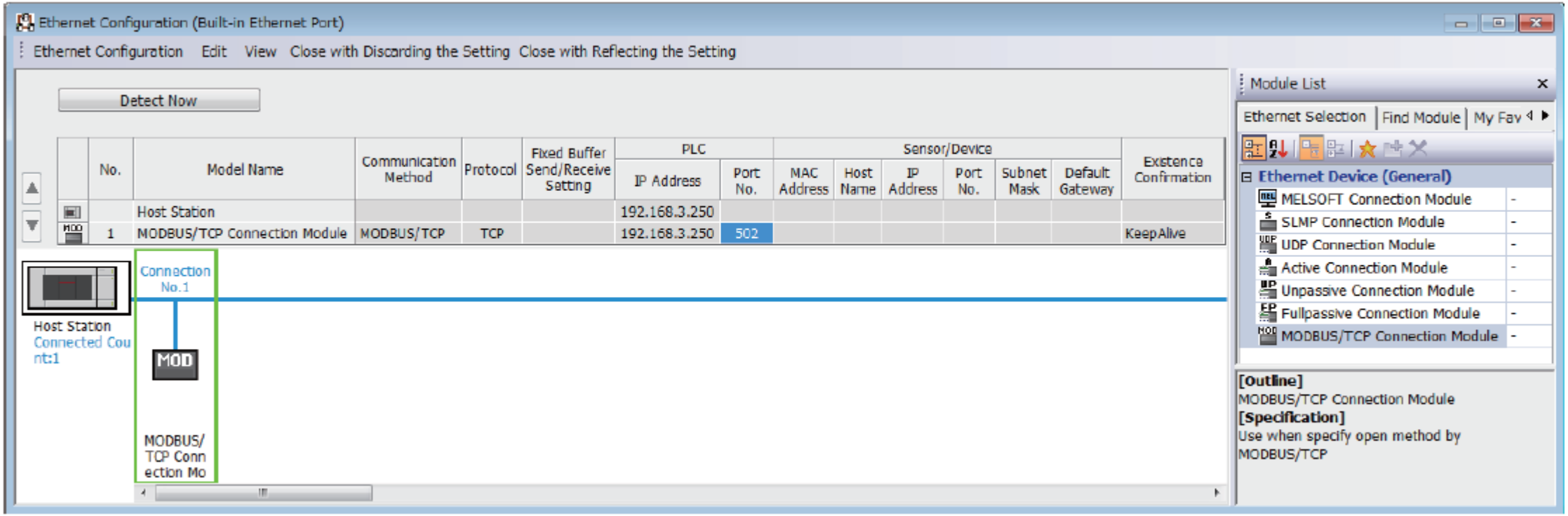
對於有高速以太網交換機保證性能的高性能內網，情況就完全不同了。理論上，Modbus TCP/IP 在批量傳輸寄存器時以高達 250/(250+70+70) 或大約 60% 的效率傳輸數據。由於 10BaseT 以太網承載約 1.25 Mbps 原始數據，理論吞吐量為每秒 1.25M/2 * 60% = 360000 個寄存器，100BaseT 速度是其 10 倍。

這假設您使用的設備能夠以與可用帶寬一樣快的速度為以太網提供服務。施耐德電氣使用帶以太網 I/O 的 MOMENTUM™ 以太網 PLC 進行的實際測試表明，每秒可以掃描多達 4000 個 I/O 基板，每個 I/O 基板具有多達 16 個 12 位模擬 I/O 或32 個離散 I/O。四個鹼基可以在一毫秒內更新。雖然這低於上面計算的理論限制，但請記住，測試設備僅使用 80186 CPU 運行，頻率為 50 MHz，有效計算能力為 3 MIPS（與 500 MHz Pentium 的 700 MIPS 相比）。然而，這些結果比迄今為止使用的專有 I/O 掃描方法更快。隨著低端 CPU 變得越來越便宜，Momentum 類型的設備將追逐理論極限，儘管它們“

https://modbus.org/faq.php

Modbus/TCP Setup

"Navigation window" ⇨[Parameter] ⇨[FX5UCPU] ⇨[Module Parameter] ⇨[Ethernet Port] ⇨[Basic Settings] ⇨ [External Device Configuration] ⇨ <Detailed Setting> of "External Device Configuration" is double-clicked



https://www.youtube.com/watch?v=Bj5gDXS3xmk&t=1s

Modbus standard functions

Function code	Function Name	Details	Accessible devices per message	Broadcast	Reference
01H	Read coils	Read binary (R/W) devices	1 to 2000 points	×	Page 66
02H	Read inputs	Read binary (RO) devices	1 to 2000 points	×	Page 67
03H	Read holding registers	Read 16 bit (R/W) registers	1 to 125 points	×	Page 68
04H	Read input registers	Read 16 bit (RO) registers	1 to 125 points	×	Page 69
05H	Write single coil	Write single binary device	1 point	○	Page 70
06H	Write single register	Write single 16 bit register device	1 point	○	Page 70
0FH	Write multiple coils	Write multiple binary (R/W) devices	1 to 1968 points	○	Page 71
10H	Write multiple registers	Write multiple 16 bit (R/W) registers	1 to 123 points	○	Page 72

Modbus Address X,Y,M,D

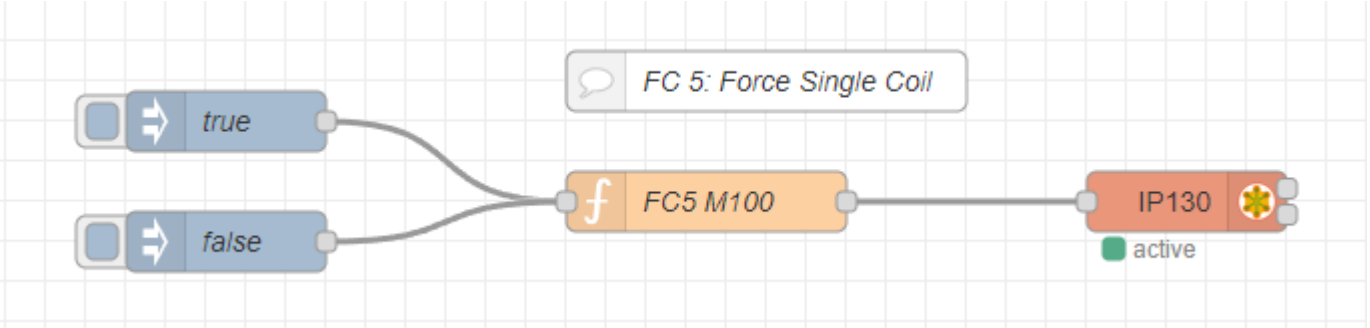
MODBUS address <Bit device>		FX5 device			
		Coil (read/write)		Input (read only)	
FX5UJ	FX5U/FX5UC	FX5UJ	FX5U/FX5UC	FX5UJ	FX5U/FX5UC
0000H to 03FFH	0000H to 03FFH	Y0 to 1023	Y0 to 1023	X0 to 1023	X0 to 1023
0400H to 1FFFH	0400H to 1FFFH	—	—	—	—
2000H to 3DFFH	2000H to 3DFFH	M0 to 7679	M0 to 7679	—	—

MODBUS address <Word device>		FX5 device			
		Input register (read only)		Holding register (read/write)	
FX5UJ	FX5U/FX5UC	FX5UJ	FX5U/FX5UC	FX5UJ	FX5U/FX5UC
0000H to 1F3FH	0000H to 1F3FH	—	—	D0 to 7999	D0 to 7999

Y0～1023　　0000H～03FFH
M0～M7679　　2000H(8192)～3DFFH(15871)

D0～D7999　　0000H～1F3FH

Modbus FC 5: Force Single Coil



名稱 FC5 M100

Setup | 設置 | 函數 | 關閉

```
1   msg.payload =
2   {
3       value: msg.payload,
4       'fc': 5, //FC 5: Force Single Coil
5       'unitid': 1,
6       'address': 8292,// (8192 + offset) M100= 8192+100=8292
7       'quantity': 1
8   }
9   return msg;
10
```

Watch 1[Watching]

ON | OFF | ON/OFF toggle | Update

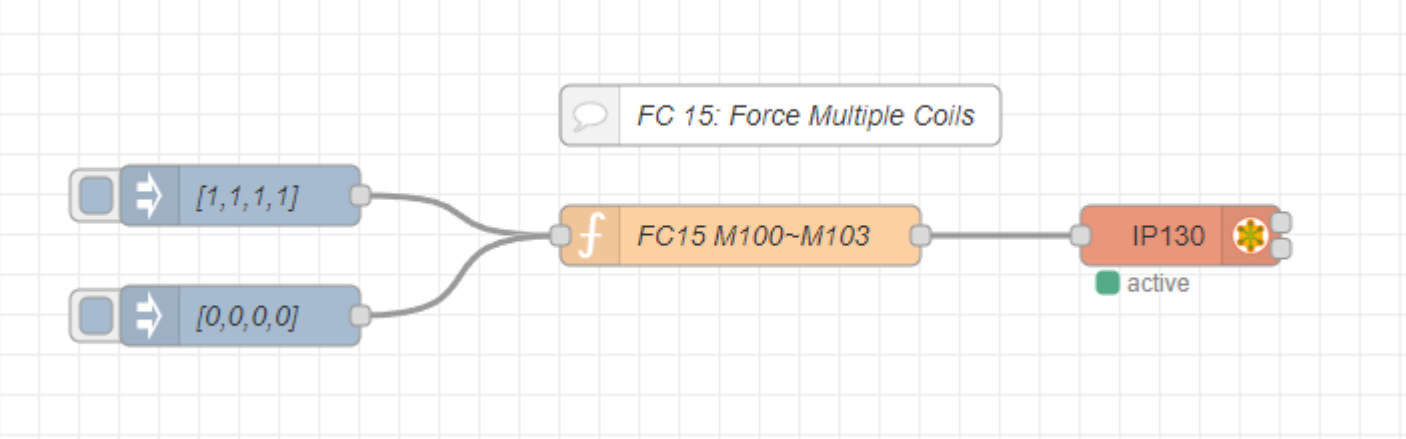
Name	Current Value	Display Format	Data Type	English
M100	TRUE	BIN	Bit	

Watch 1[Watching]

ON | OFF | ON/OFF toggle | Update

Name	Current Value	Display Format	Data Type	English
M100	FALSE	BIN	Bit	

Modbus FC 15: Force Multiple Coils



名稱 FC15 M100~M103

Setup | 設置 | 函數 | 關閉

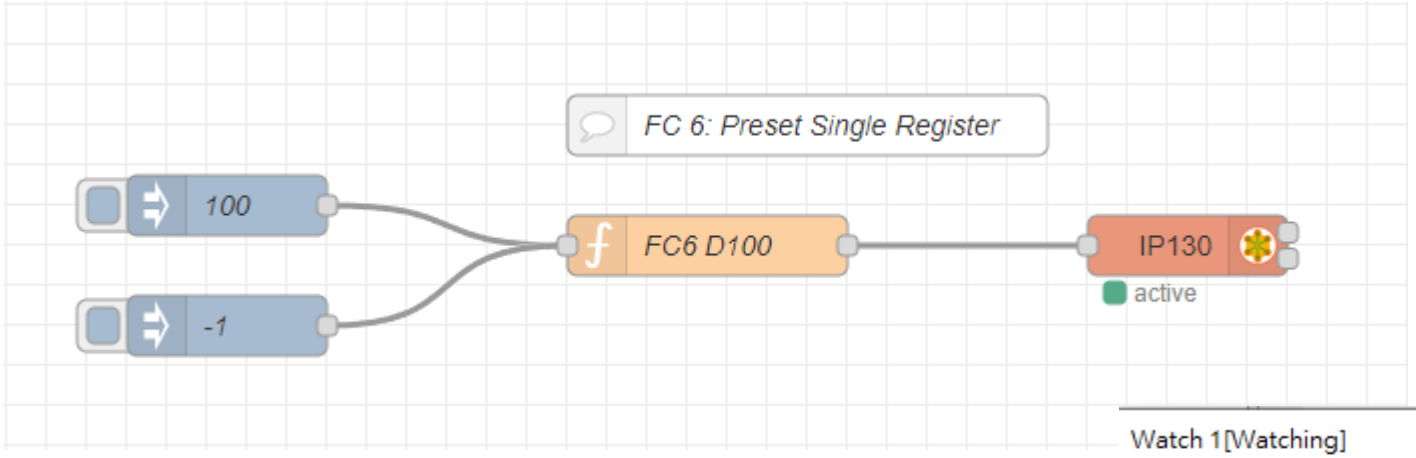
```
msg.payload =
{
    value: msg.payload,
    'fc': 15, //FC 15: Force Multiple Coils
    'unitid': 1,
    'address': 8292,// (8192 + offset) M100= 8192+100=8292
    'quantity': 4
}
return msg;
```

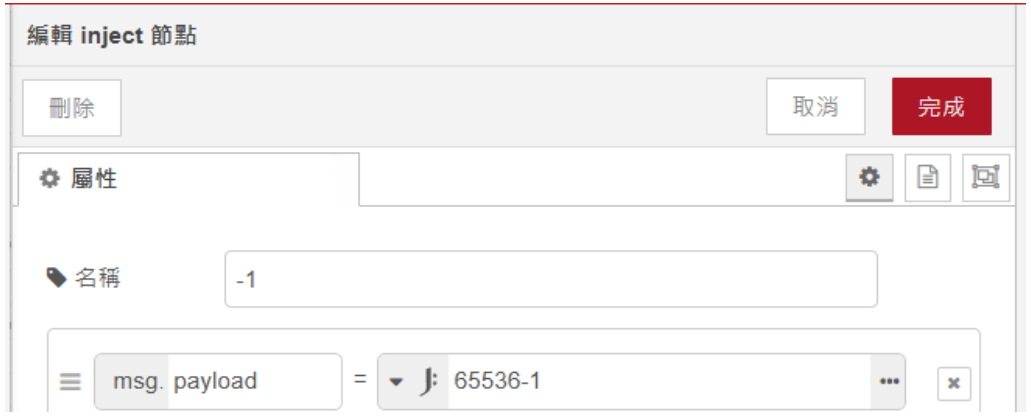
Watch 1[Watching]

ON | OFF | ON/OFF toggle | Update

Name	Current Value	Display Format	Data Type
M100	TRUE	BIN	Bit
M101	TRUE	BIN	Bit
M102	TRUE	BIN	Bit
M103	TRUE	BIN	Bit
D100	105	Decimal	Word [Signed]
D101	55	Decimal	Word [Signed]
D102	30	Decimal	Word [Signed]
D103	15	Decimal	Word [Signed]

Modbus FC 6: Preset Single Register





名稱 FC6 D100

Setup | 設置 | 函數 | 關閉

```
1   msg.payload =
2   {
3       value: msg.payload,
4       'fc': 6, //FC 6: Preset Single Register
5       'unitid': 1,
6       'address': 100, // D0 = 0
7       'quantity': 1
8   }
9   return msg;
10
```

Watch 1[Watching]

ON | OFF | ON/OFF toggle | Update

Name	Current Value	Display Format	Data Type	English
D100	-1	Decimal	Word [Signed]	

Watch 1[Watching]

ON | OFF | ON/OFF toggle | Update

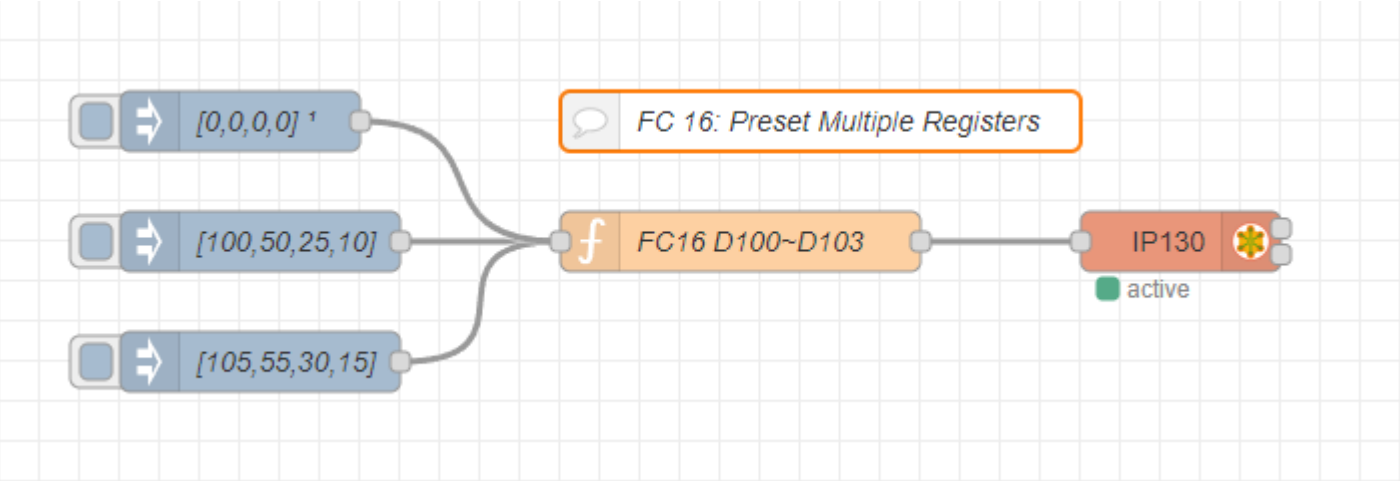
Name	Current Value	Display Format	Data Type	English
D100	HFFFF	Hexadecimal	Word [Signed]	

Watch 1[Watching]

ON | OFF | ON/OFF toggle | Update

Name	Current Value	Display Format	Data Type	English
D100	65535	Decimal	Word [Unsigned]/Bit String [16-bit]	

Modbus FC 16: Preset Multiple Registers



名稱 FC16 D100~D103

Setup | 設置 | 函數 | 關閉

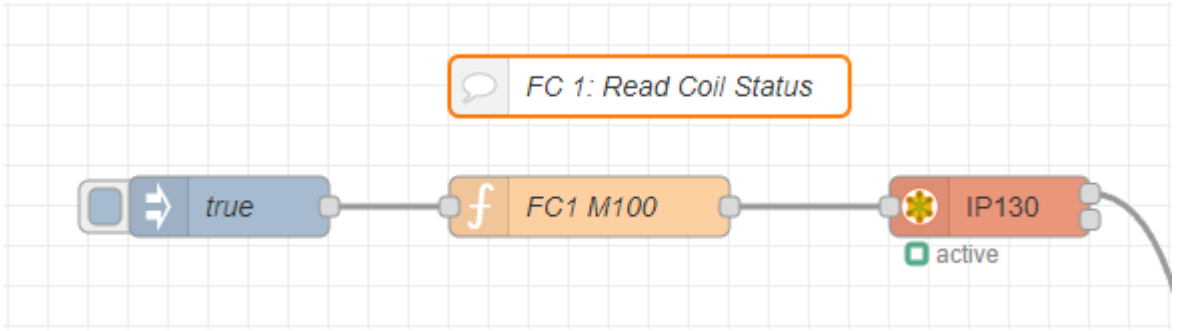
```
1   msg.payload =
2   {
3       value: msg.payload,
4       'fc': 16, //FC 16: Preset Multiple Registers
5       'unitid': 1,
6       'address': 100,// D0 = 0
7       'quantity': 4
8   }
9   return msg;
10
```

Watch 1[Watching]

ON | OFF | ON/OFF toggle | Update

Name	Current Value	Display Format	Data Type
M100	TRUE	BIN	Bit
M101	TRUE	BIN	Bit
M102	TRUE	BIN	Bit
M103	TRUE	BIN	Bit
D100	105	Decimal	Word [Signed]
D101	55	Decimal	Word [Signed]
D102	30	Decimal	Word [Signed]
D103	15	Decimal	Word [Signed]

Modbus FC 1: Read Coil Status



cd9207688f98a990 : msg.payload : array[8]

▸ [true, false, false, false, false, false, false, false]

名稱 FC1 M100

Setup | 設置 | 函數 | 關閉

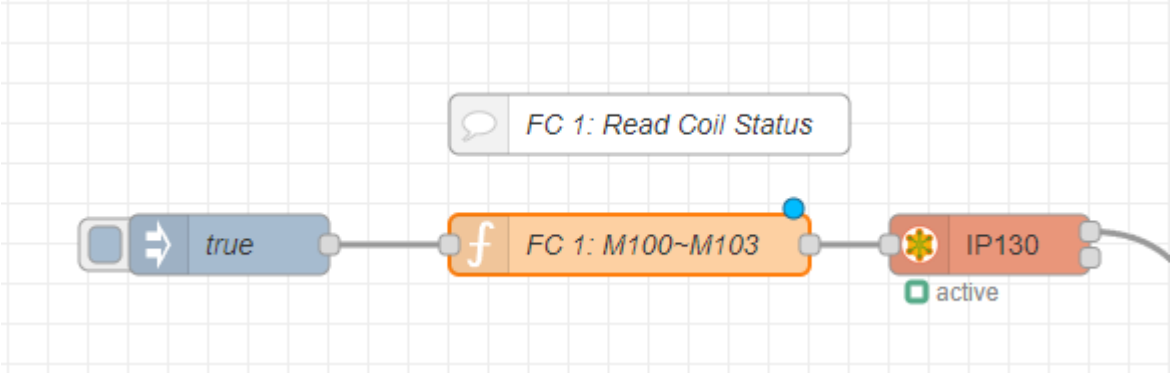
```
1   msg.payload =
2   {
3       value: msg.payload,
4       'fc': 1, //FC 1: Read Coil Status
5       'unitid': 1,
6       'address': 8292,// M0 = 8192
7       'quantity': 1
8   }
9   return msg;
10
```

Watch 1[Watching]

ON | OFF | ON/OFF toggle | Update

Name	Current Value	Display Format	Data Type
M100	TRUE	BIN	Bit
M101	TRUE	BIN	Bit
M102	TRUE	BIN	Bit
M103	TRUE	BIN	Bit
D100	105	Decimal	Word [Signed]
D101	55	Decimal	Word [Signed]
D102	30	Decimal	Word [Signed]
D103	15	Decimal	Word [Signed]

Modbus FC 1: Read Coil Status




```
24b9525ca2d3f644 : msg.payload : array[8]
▸ [ true, true, true, true, false,
false, false, false ]
```

名稱 FC 1: M100~M103

Setup | 設置 | 函數 | 關閉

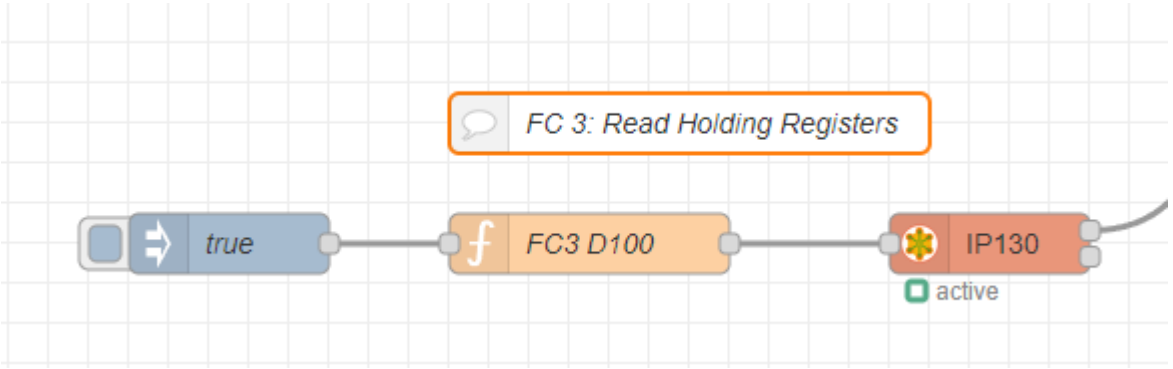
```
1   msg.payload =
2   {
3       value: msg.payload,
4       'fc': 1, //FC 1: Read Coil Status
5       'unitid': 1,
6       'address': 8292,// M0 = 8192
7       'quantity': 4
8   }
9   return msg;
10
```

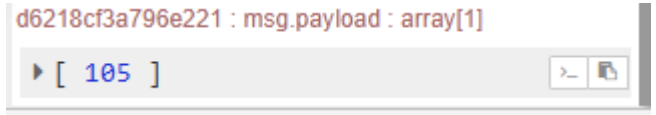
Watch 1[Watching]

ON | OFF | ON/OFF toggle | Update

Name	Current Value	Display Format	Data Type
M100	TRUE	BIN	Bit
M101	TRUE	BIN	Bit
M102	TRUE	BIN	Bit
M103	TRUE	BIN	Bit
D100	105	Decimal	Word [Signed]
D101	55	Decimal	Word [Signed]
D102	30	Decimal	Word [Signed]
D103	15	Decimal	Word [Signed]

Modbus FC 3: Read Holding Registers





名稱 FC3 D100

Setup | 設置 | 函數 | 關閉

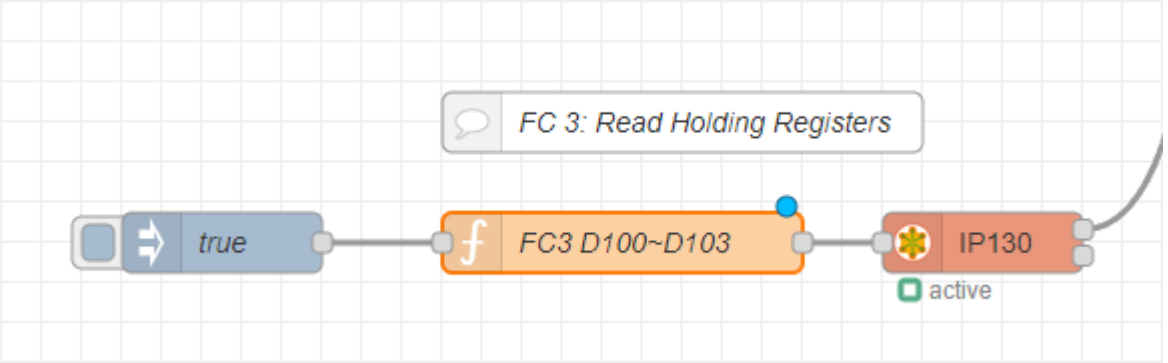
```
1   msg.payload =
2   {
3       value: msg.payload,
4       'fc': 3, //FC 3: Read Holding Registers
5       'unitid': 1,
6       'address': 100, // D0 = 0
7       'quantity': 1
8   }
9   return msg;
10
```

Watch 1[Watching]

ON | OFF | ON/OFF toggle | Update

Name	Current Value	Display Format	Data Type
M100	TRUE	BIN	Bit
M101	TRUE	BIN	Bit
M102	TRUE	BIN	Bit
M103	TRUE	BIN	Bit
D100	105	Decimal	Word [Signed]
D101	55	Decimal	Word [Signed]
D102	30	Decimal	Word [Signed]
D103	15	Decimal	Word [Signed]

Modbus FC 3: Read Holding Registers



5d1389c013c1b07c : msg.payload : array[4]

▸ [105, 55, 30, 15]

名稱 FC3 D100~D103

Setup | 設置 | 函數 | 關閉

```
1   msg.payload =
2   {
3       value: msg.payload,
4       'fc': 3, //FC 3: Read Holding Registers
5       'unitid': 1,
6       'address': 100,// D0 = 0
7       'quantity': 4
8   }
9   return msg;
10
```

Watch 1[Watching]

ON | OFF | ON/OFF toggle | Update

Name	Current Value	Display Format	Data Type
M100	TRUE	BIN	Bit
M101	TRUE	BIN	Bit
M102	TRUE	BIN	Bit
M103	TRUE	BIN	Bit
D100	105	Decimal	Word [Signed]
D101	55	Decimal	Word [Signed]
D102	30	Decimal	Word [Signed]
D103	15	Decimal	Word [Signed]

FX5U TON,CTU -ST

```
1
2  IF SM402 THEN
3      tT0_SV := T#1S; //initial value
4      iC0_SV := 1;
5  END_IF;
6
7  tonT0(IN:= M100 ,PT:= tT0_SV ,Q=> M101 ,ET=> tT0 );
8
9
10 ctuC0(CU:= M102 ,R:= M103 ,PV:=iC0_SV ,Q=> M104 ,CV=> iC0 );
11
```

Watch 1(M104)[Watching]

ON | OFF | ON/OFF toggle | Update

Name	Current Value	Display Format	Data Type
M100	FALSE	BIN	Bit
M101	FALSE	BIN	Bit
M102	FALSE	BIN	Bit
M103	FALSE	BIN	Bit
M104	FALSE	BIN	Bit

Global [Global Label Setting]

<Filter> | Easy Display | Display Setting | Check

	Label Name	Data Type		Class	Assign (Device/Label)
1	tT0_SV	Time	...	VAR_GLOBAL	D100
2	tT0	Time	...	VAR_GLOBAL	D102
3	iC0_SV	Word [Signed]	...	VAR_GLOBAL	D104
4	iC0	Word [Signed]	...	VAR_GLOBAL	D106
5			...		

Watch 2[Watching]

ON | OFF | ON/OFF toggle | Update

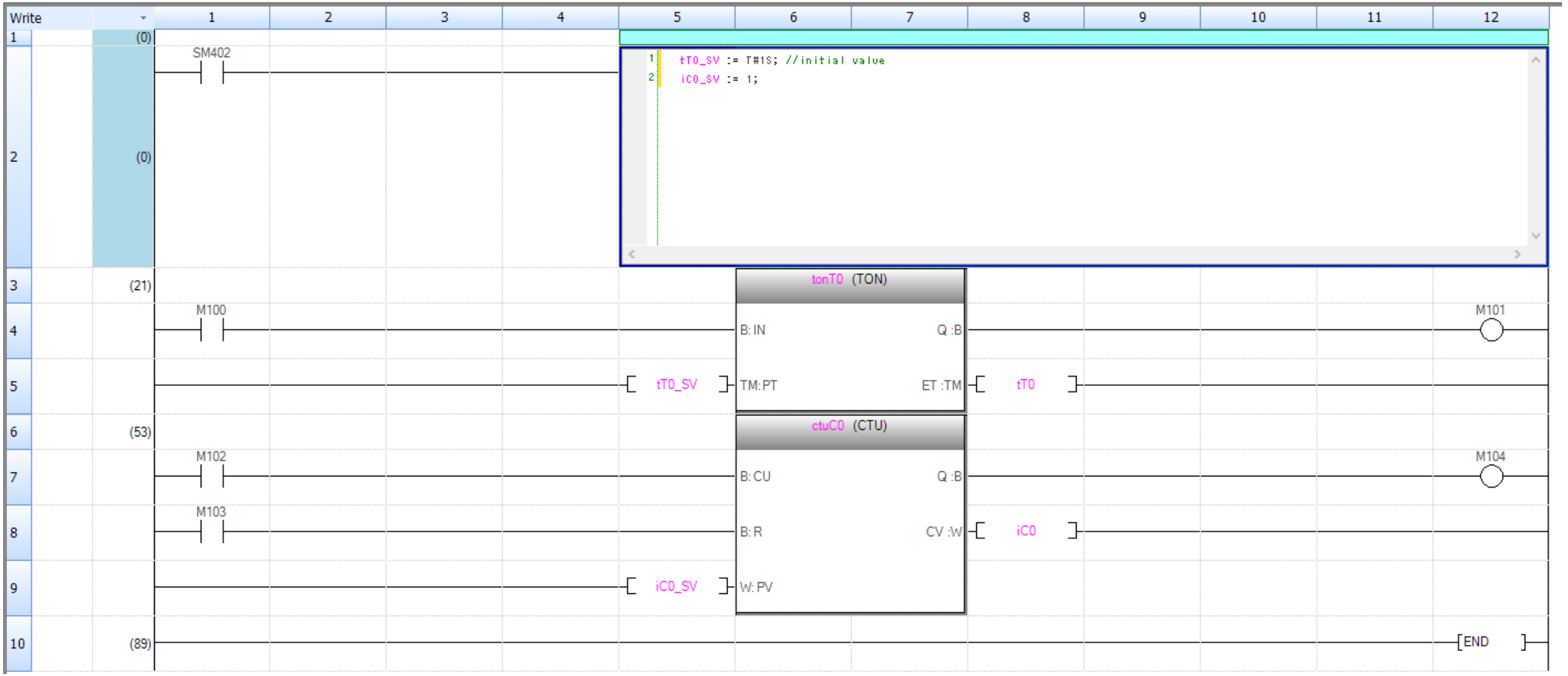
Name	Current Value	Display Format	Data Type
D100	2700	Decimal	Word [Signed]
D101	0	Decimal	Word [Signed]
D102	0	Decimal	Word [Signed]
D103	0	Decimal	Word [Signed]
D104	4	Decimal	Word [Signed]
D105	0	Decimal	Word [Signed]
D106	0	Decimal	Word [Signed]
D107	0	Decimal	Word [Signed]

ProgPou1 [PRG] [Local Label Setting]

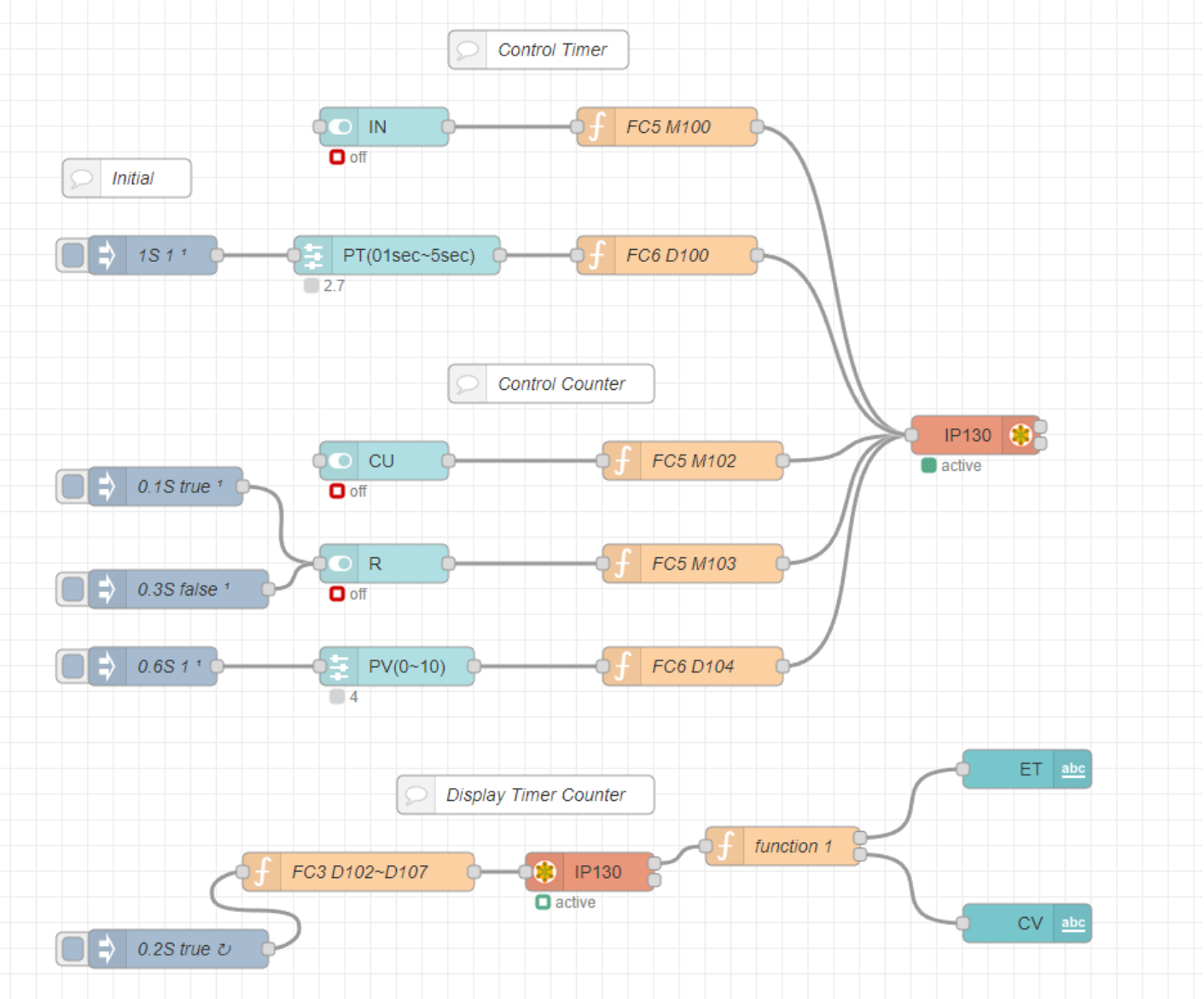
<Filter> | Show Details(Y) | Display Settin

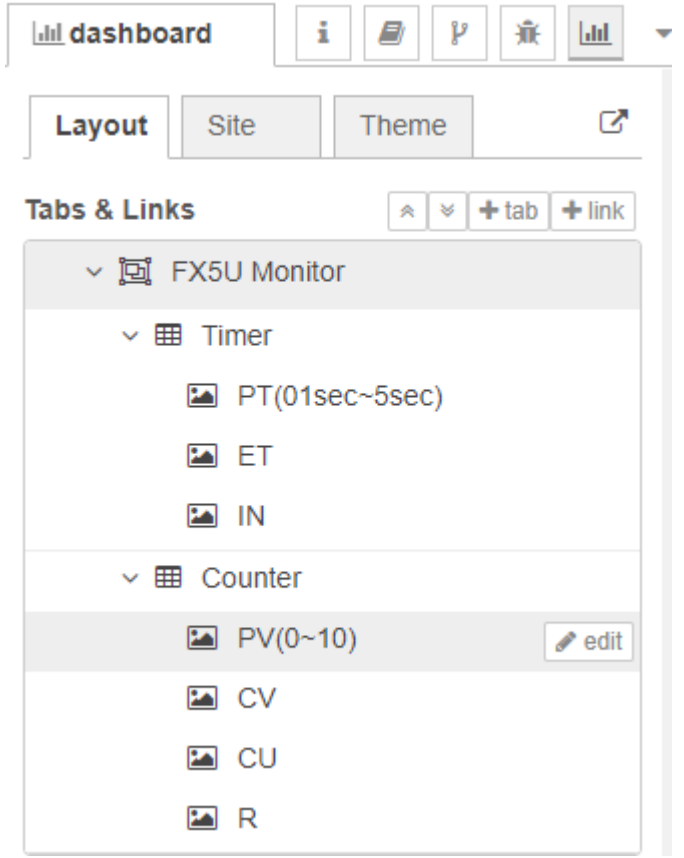
	Label Name	Data Type	
1	tonT0	TON	...
2	ctuC0	CTU	...
3			...

FX5U TON,CTU -LD



FX5U TON,CTU R/W -NR



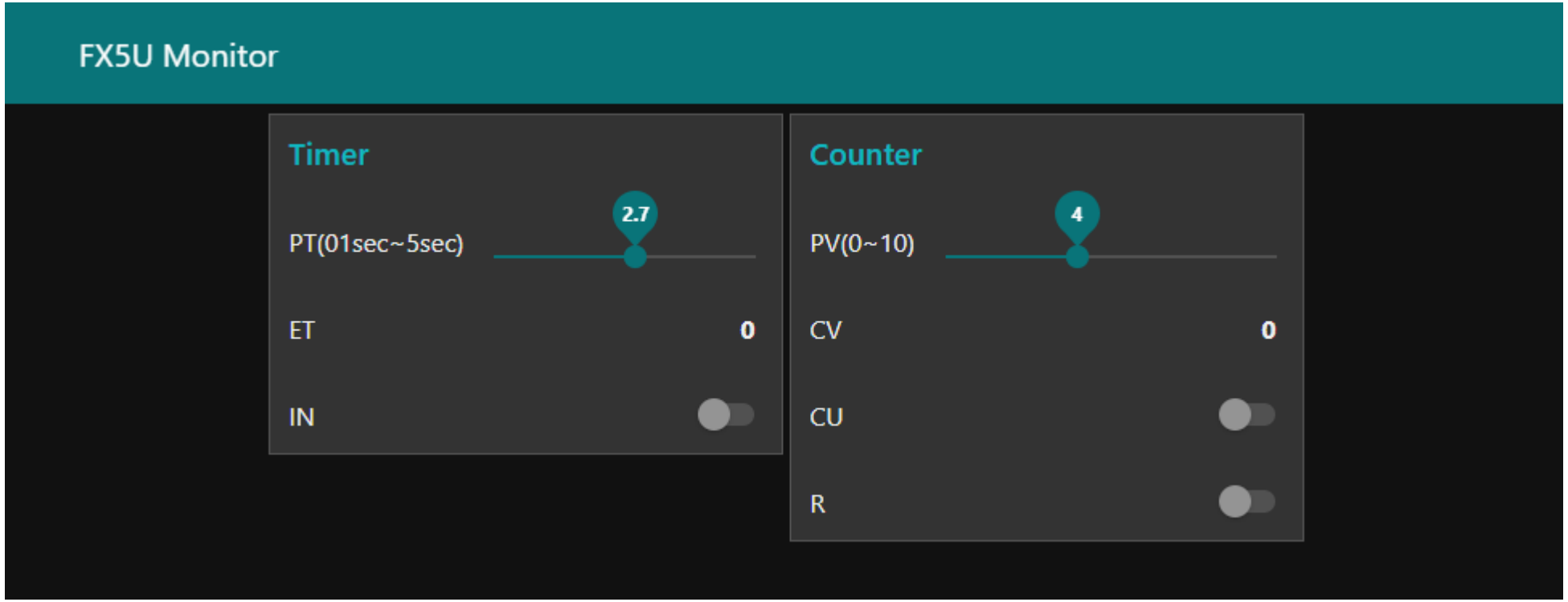


FX5U TON,CTU R/W -NR

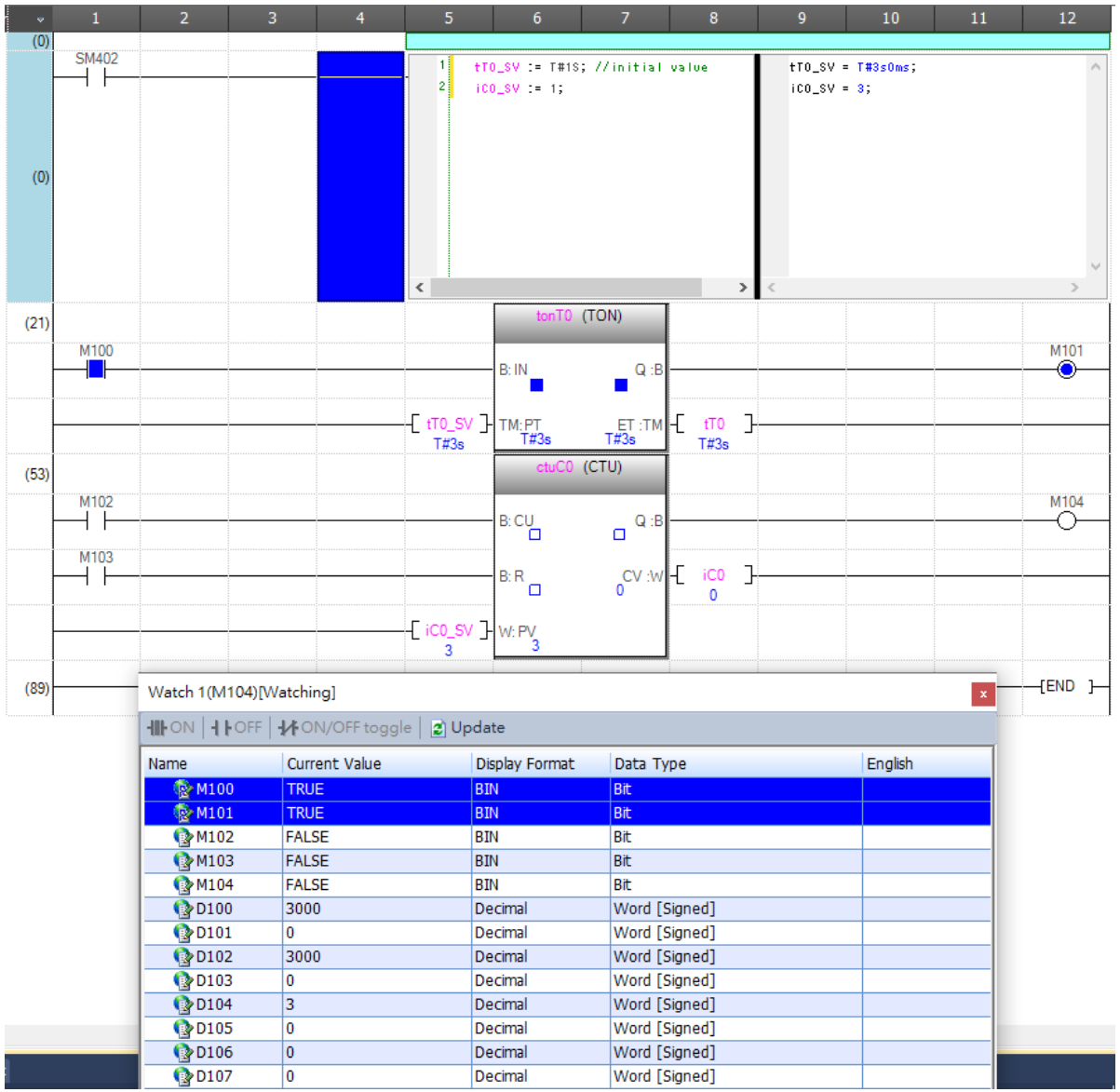


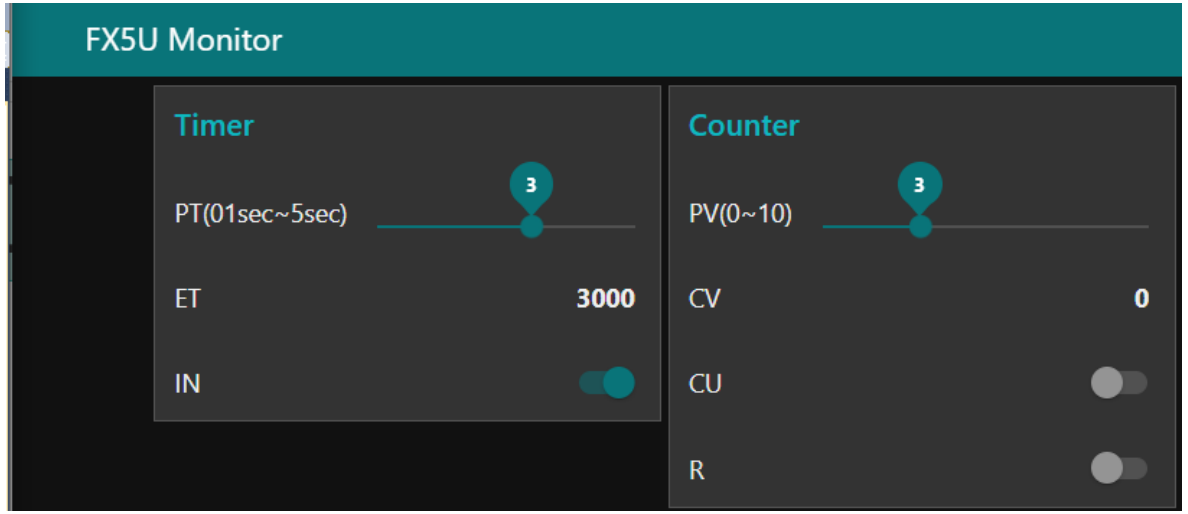


FX5U TON,CTU R/W -UI

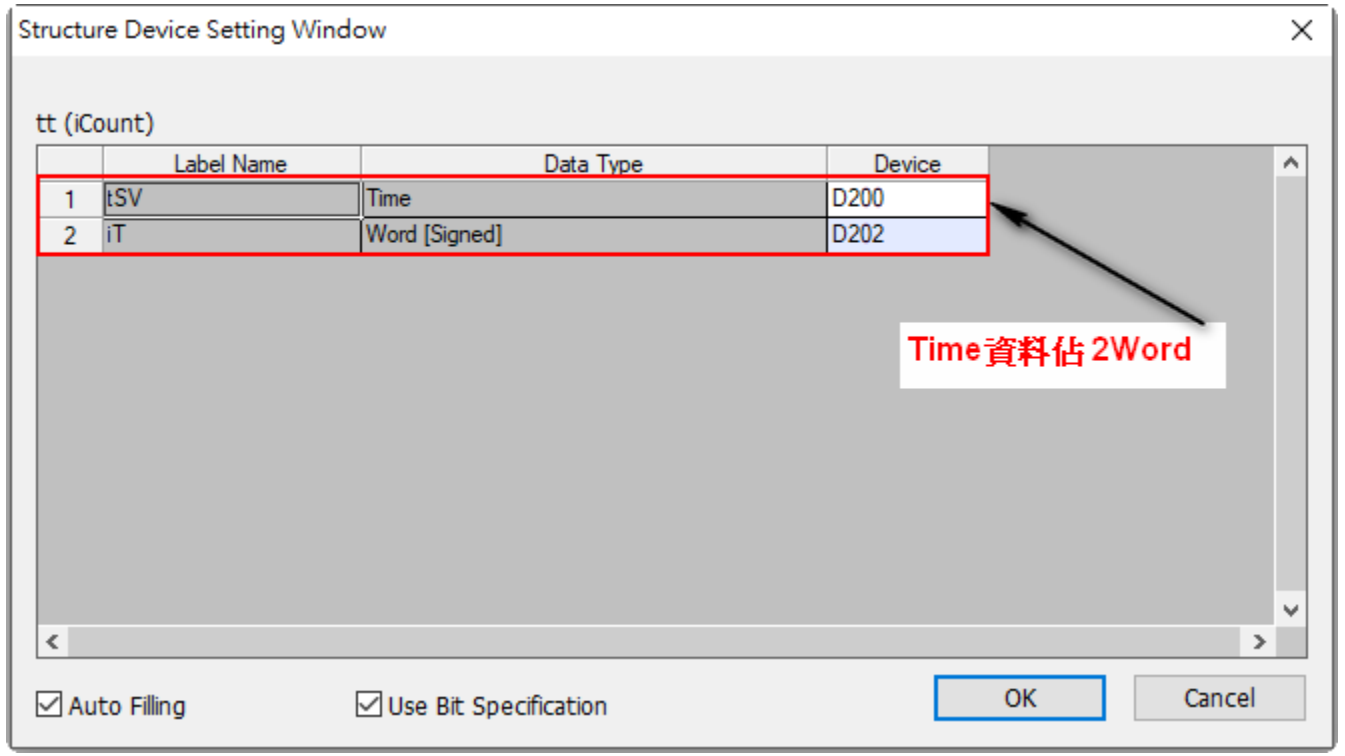


FX5U TON,CTU R/W -Monitor

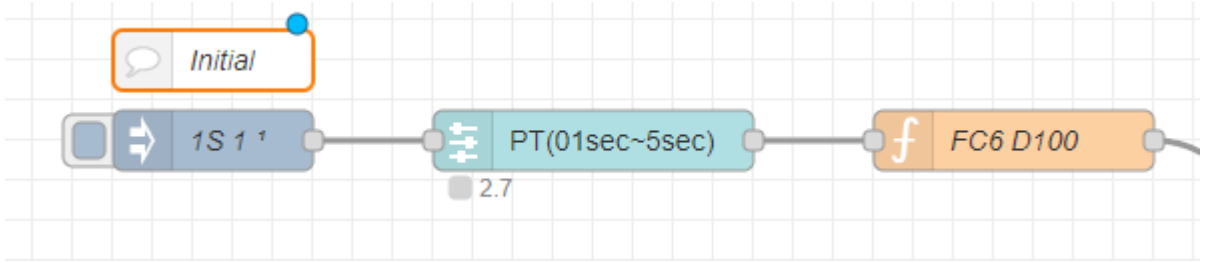




通訊位址指定


Structure Device Setting Window
tt (iCount)
Label Name
Data Type
Device
1
tSV
Time
D200
2
iT
Word [Signed]
D202
Time資料佔2Word
Auto Filling
Use Bit Specification
OK
Cancel

Dashboard Initial



立刻執行於 1 秒後, 此後

重複 無

Range min 0 max 5 step 0.1

Output only on release

If msg arrives on input, pass through to output:

When changed, send:

Payload Current value

Topic msg. topic

Name